CHMLTECH ONLINE JOURNAL · VOLUME 105, ISSUE 5 · RESEARCH ARTICLE

Agent Security and Identity: Prompt Injection, Least-Privilege Tool Access, Agent Credentials and Audit Trails

Quak Foo Lee
Chief Technology Officer, Chmltech Ltd.
Division of Artificial Intelligence and Machine Learning · Vancouver, BC, Canada

Published November 12, 2026 14 min read

ABSTRACT

An AI agent that can call tools acts with real authority: it reads records, moves data and changes systems. Its security therefore rests less on the model than on the identity it acts under and the permissions attached to that identity. This paper sets out a practical security model for tool-using agents in small and mid-sized firms. It first describes the threat model, centred on prompt injection, the lethal trifecta of private data, untrusted content and external communication, and the ten risk categories of the OWASP Top 10 for Agentic Applications 2026, and shows with a simple probability model why filtering alone cannot make an exposed agent safe. It then treats the agent as a first-class non-human identity with an accountable owner, delegated rather than shared credentials, and short-lived, audience-bound tokens, drawing on the Model Context Protocol authorization model and the NIST NCCoE concept paper on agent identity and authorization. It describes least-privilege tool access, runtime checkpoints, human approval for irreversible actions, agent designs that resist injection, and the audit record each agent action should leave, and closes with a governance lifecycle and a ten-point control checklist. The central argument is that agent identity and

governance, not prompt wording, are the dependable control point: instructions can be subverted, but a credential that does not grant an action cannot be talked into granting it.

Keywords: agent security; prompt injection; least privilege; non-human identity; OAuth; audit trail; AI governance; Model Context Protocol

1. Introduction: Identity as the Control Point

Security is already the most common reason firms hold back on AI. In Statistics Canada's second-quarter 2026 survey, cybersecurity or privacy concerns were the barrier businesses cited most often, ahead of cost; among firms with 100 or more employees, 30.0% named it [1]. Buyers are responding: in Jitterbit's survey of more than 1,500 businesses, 47% ranked security, auditability, traceability and guardrails as a top criterion when choosing AI tools [2]. Gartner lists inadequate risk controls among the three reasons it expects more than 40% of agentic AI projects to be cancelled by the end of 2027 [3].

What makes agents different from earlier AI is that they act. A chat assistant that gives a bad answer wastes a reader's time. An agent with a bad instruction can send an email, approve a credit or delete a file, using whatever access it holds. That access is granted through identities and credentials, which is why identity and access management (IAM) vendors now describe agent identity as the control plane of the agentic enterprise [4], and why the U.S. National Institute of Standards and Technology (NIST) opened a project in February 2026 on applying identity standards to software and AI agents [5, 6].

The argument of this paper is simple. Model-level defences against manipulation help, but they are probabilistic. Identity and authorization controls are deterministic: if a credential does not permit an action, no prompt can make the downstream system accept it. A sound agent security design therefore starts from who the agent is, whom it acts for and what it is allowed to touch, and only then adds model-level guardrails. Sections 2 to 6 cover the threat model, identity and credentials, least-privilege tool access, audit trails and governance.

2. Threat Model for Tool-Using Agents

2.1 Prompt injection

Prompt injection occurs when text the agent reads, rather than text its operator wrote, changes what the agent does. The root cause is architectural: a language model receives instructions and data in the same stream of tokens and cannot reliably tell which source an instruction came from [7]. Any web page, email, PDF, ticket or database field the agent reads can carry hidden instructions. Willison, who coined the term, notes that vendors have fixed many specific exploits, but that users who combine tools themselves cannot rely on vendors to protect them [7].

2.2 The lethal trifecta

Willison identifies three capabilities that are dangerous in combination [7]:

1. **Access to private data** (the ERP, the CMMS, email, shared drives).
2. **Exposure to untrusted content** (supplier emails, customer tickets, web pages, uploaded documents).
3. **Ability to communicate externally** (sending email, calling an external interface, even rendering a link or image that fetches a URL).

An agent with all three can be instructed by an attacker, through content it reads, to collect private data and send it out. The dependable defence is to remove one leg of the trifecta for any given agent or task.

Industrial example. A purchasing agent reads supplier quotes from a shared inbox (untrusted content), looks up contract prices in the ERP (private data) and can email suppliers (external communication). A quote PDF carrying hidden text that asks the agent to send the full price list to an outside address meets every condition. Splitting the work so that the agent which reads incoming mail cannot send mail, and the agent that sends mail only sends drafts a person approved, breaks the chain (Figure 1).

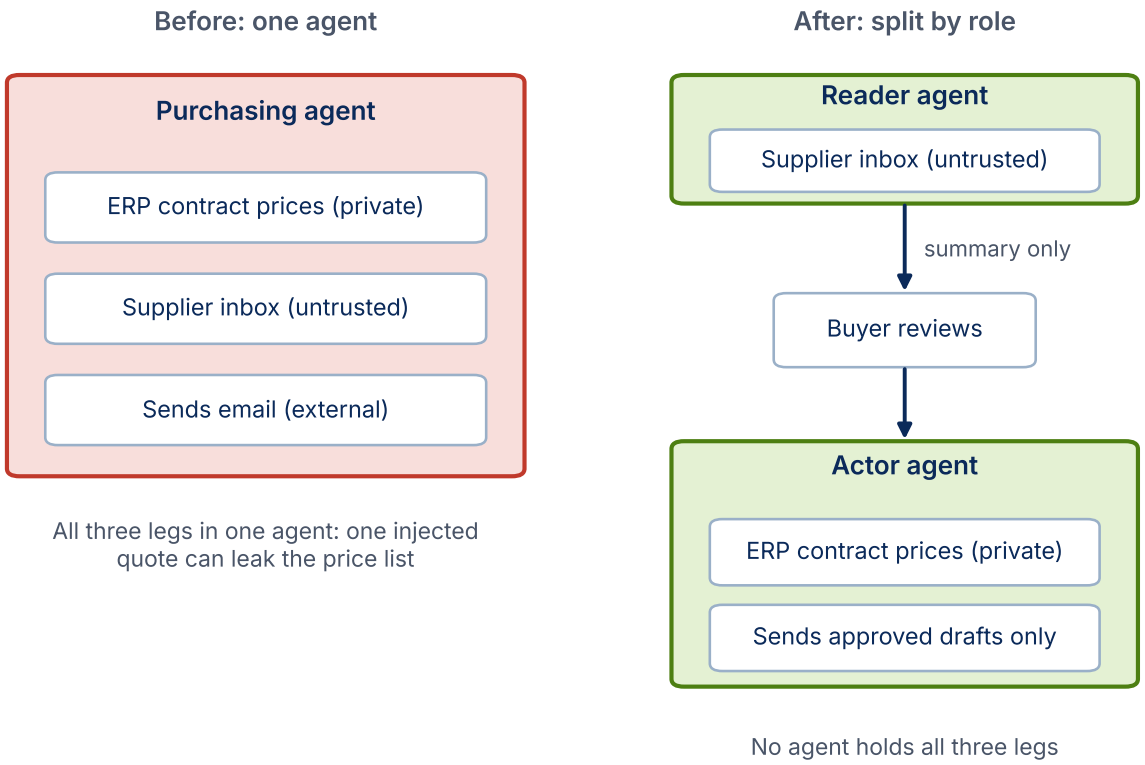


Figure 1. Breaking the lethal trifecta in the purchasing example. Before, one agent holds private data, untrusted content and external communication. After, a reader agent sees untrusted content but can only pass a summary to a person, and the actor agent holds private data and sends only approved drafts.

Drawn by the author after Willison [7].

2.3 Why filtering alone is not enough

Injection filters and model-level defences are probabilistic. If each attempt succeeds with probability p and an attacker can make n independent attempts (by sending many emails, documents or tickets), the probability that at least one succeeds is given by Eq. (1):

$$P_{\text{breach}} = 1 - (1 - p)^{n} \tag{1}$$

Worked example (illustrative figures). A filter that stops 95% of injection attempts leaves p = 0.05. An attacker who can place 20 crafted documents in the agent's path has a $1 - 0.95^{20} \approx 64\%$ chance of at least one success; with 100 attempts the chance exceeds 99%. Sending documents to a supplier or customer inbox costs an attacker almost nothing, so n is large in practice. This is why Willison describes a filter that stops 95% of attacks as a failing grade by

web-security standards [7], and why the controls in Sections 3 and 4 aim to make a successful injection harmless rather than merely unlikely.

2.4 The OWASP Top 10 for Agentic Applications

On 9 December 2025 the OWASP GenAI Security Project published its first Top 10 for Agentic Applications, built from observed incidents with input from more than 100 security practitioners [8, 9]. It introduces the principle of least agency: give an agent only the autonomy a bounded task requires [8]. Table 1 lists the ten categories with industrial examples.

Table 1. OWASP Top 10 for Agentic Applications 2026, with industrial examples and primary controls.

ID	Risk (OWASP)	What it looks like in an industrial firm	Primary control
ASI01	Agent goal hijack	A work-request note redirects a maintenance agent's objective	Treat retrieved content as data; constrain objectives
ASI02	Tool misuse and exploitation	Agent uses a legitimate ERP tool with harmful parameters	Narrow tools; parameter validation
ASI03	Identity and privilege abuse	Agent reuses an administrator's inherited session	Per-agent identity; short-lived scoped tokens
ASI04	Agentic supply-chain vulnerabilities	A third-party MCP server ships a poisoned tool description	Approved server registry; version pinning
ASI05	Unexpected code execution	Agent runs a generated script on a production server	Sandboxed execution; review before run
ASI06	Memory and context poisoning	Bad data written to agent memory or a retrieval index skews later answers	Write controls on memory and indexes
ASI07	Insecure inter-agent communication	A spoofed agent message triggers a schedule change	Authenticated agent-to-agent channels
ASI08	Cascading failures	One agent's wrong belief propagates through a workflow	Checkpoints between agents; circuit breakers
ASI09	Human-agent trust exploitation	Operators approve agent proposals without reading them	Clear approval prompts; sampling reviews
ASI10	Rogue agents	A compromised agent acts while appearing normal	Behavioural monitoring; kill switch

Sources: risk names from OWASP as reported in [8, 9]; industrial examples and control wording by the author.

2.5 Protocol-level threats

The MCP specification documents threats specific to tool connections. Two matter most for integrators [10]. In a confused-deputy attack, an MCP server that brokers access to a third-party service is tricked into using its own authority for an attacker. In token passthrough, a server accepts a token that was not issued to it and forwards it downstream, bypassing audience checks, rate limits and logging; the specification forbids this. The 2026-07-28 revision adds validation of the token issuer to defend against OAuth mix-up attacks when one client uses several servers [11].

3. Agent Identity and Credentials

3.1 The agent as a first-class identity

Most early agent deployments ran under a developer's personal account or a shared service account with a long-lived key. Both hide who did what. The NIST National Cybersecurity Center of Excellence (NCCoE) concept paper, released on 5 February 2026, frames the problem in the standard IAM terms of identification, authentication, authorization, auditing and non-repudiation, and asks how credentials for agents should be issued, rotated and revoked, and whether authorization can adapt as an agent's context changes [5, 6, 12]. Its stated aim is to adapt existing identity standards rather than invent new ones [12]. In practice each agent should have:

- **Its own registered identity** in the corporate directory, as a workload or service principal, distinct from any person.
- **A named, accountable human owner** who answers for its permissions and behaviour.
- **A declared purpose** that bounds which tools and data it may use.
- **A lifecycle**: requested, approved, deployed, reviewed and retired, like any other privileged account.

3.2 Delegation, not impersonation

When an agent acts for a user, the downstream system should see both identities: the agent and the user it acts for. Standard OAuth delegation, including the token exchange defined in RFC 8693, can carry both in a token, so the ERP can apply the user's permissions and the log can record the agent [6, 13]. The anti-pattern is the agent simply holding a user's password or

session, which makes its actions indistinguishable from the user's and gives it all of the user's rights, not only those the task needs. For MCP specifically, the rules are [10, 11]:

- A server accepts only tokens issued for that server (audience-bound), and never forwards a client's token downstream.
- Each server obtains its own narrowly scoped credential for each downstream system.
- Clients validate the issuer in authorization responses, following RFC 9207 [14].
- Where possible, the enterprise-managed authorization extension is used, so that access to MCP servers is granted and revoked centrally through the corporate identity provider.

3.3 Short-lived credentials

Credential lifetime sets the worst case. If a credential leaks, the window W in which it can be misused is bounded by its own lifetime and by how fast the leak is detected and the credential revoked, as in Eq. (2):

$$W = \min(T_{token}, T_{detect} + T_{revoke}) \tag{2}$$

Worked example (illustrative figures). A static key rotated every 90 days has T_{token} = 129,600 minutes. If detection takes a week, W is about 10,080 minutes. A token issued per task with a 15-minute lifetime caps W at 15 minutes whatever the detection time, a reduction of roughly 670 times against the one-week case. Short-lived, per-task tokens also make revocation almost automatic: an agent that is switched off simply stops receiving new ones.

3.4 Secrets hygiene

Credentials never belong in prompts, tool descriptions, agent memory or retrieved documents, because anything in the model's context can be repeated in its output or exfiltrated through an injection. Secrets should live in a vault and be attached to calls by the gateway or server, outside the model's view.

3.5 Agent identity in a small firm

None of this requires an enterprise identity platform. Most small and mid-sized firms already run a cloud directory for email and office software, and that directory can hold agent identities

as application or service registrations, each with its own credentials, an owner field and a review date. The ERP's existing role model then does most of the authorization work: an agent's identity is given a role with only the transactions its purpose requires, in the same way a new employee would be given a role, and the agent acts for a user only through delegated tokens rather than through that user's password. The practical sequence is short. Register each agent in the directory before it is connected to anything; give it a role in each target system, starting from read-only; issue credentials through the gateway rather than to the agent's code; and record the owner and review date in the agent register described in Section 6.1. A firm that does these four things has already closed the most common gaps, which are shared keys, personal accounts and agents that nobody remembers approving.

4. Least-Privilege Tool Access and Runtime Guardrails

Least privilege for agents means limiting three things at once: which tools an agent can call, what each call can do, and how much it can do without a person in the loop. OWASP's least-agency principle extends the classic rule from permissions to autonomy [8].

4.1 Scope the tools

- **Per-agent allow-lists.** Each agent sees only the tools its purpose requires. A quality agent that summarizes deviations has no reason to see the purchasing tools.
- **Narrow, typed tools.** A tool such as get_batch_record(batch_id) is preferable to a general query tool. Every parameter is validated against its schema and business rules (ranges, units, allowed values) before the call reaches the system.
- **Row- and field-level limits.** Where the target system supports it, the delegating user's own data permissions apply to the agent's call.
- **Break the trifecta by design.** Agents are split so that no single agent both reads untrusted content and can communicate externally while holding private data (Section 2.2).

4.2 Check every call at run time

Figure 2 shows the checkpoints a gateway applies to each tool call. Identity and policy checks are deterministic; human approval is reserved for calls classed as high-risk or irreversible.

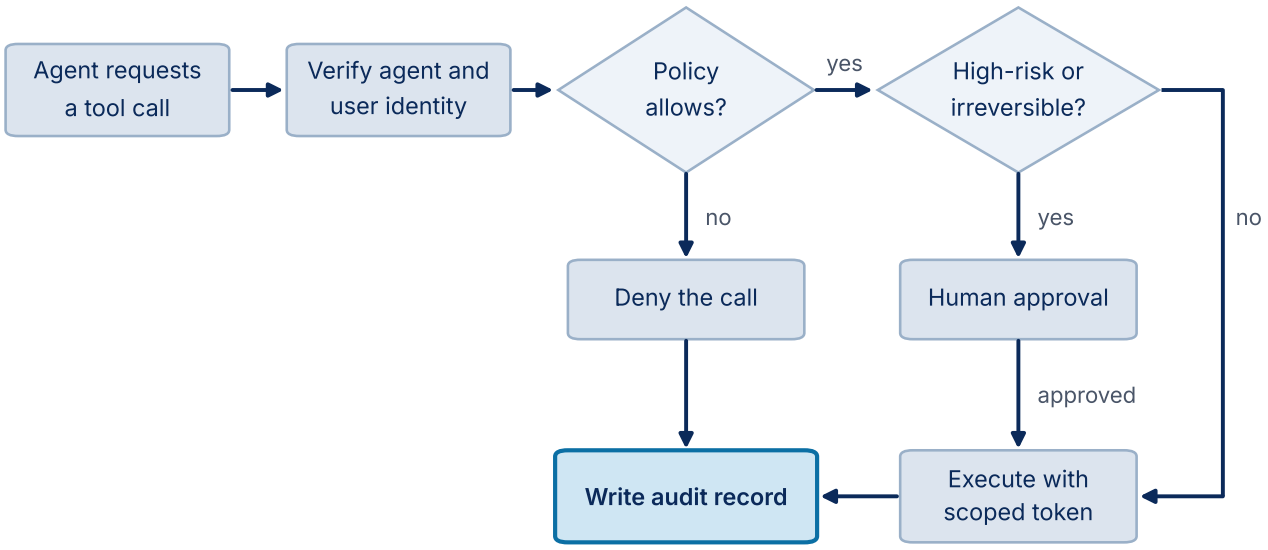


Figure 2. Runtime checkpoints on a single agent tool call. Identity and policy checks are deterministic, human approval is reserved for calls classed as high-risk or irreversible, and every path ends in the audit record. The branch for a rejected approval is omitted for clarity; it also ends in the audit record.

Drawn by the author.

Table 2 gives a practical risk tiering for an industrial firm.

Table 2. Risk tiers for agent tool calls in an industrial firm.

Tier	Examples	Default handling
Read	Order status, historian trends, SOP lookup	Automatic, logged
Reversible write	Draft requisition, maintenance notification, CRM note	Automatic, logged, sampled for review
Consequential write	Release a purchase order, change a customer credit limit, send external email	Human approval each time
Prohibited	Change a control setpoint, delete records, alter user permissions	Not exposed as a tool

Author's classification.

4.3 Contain what the agent can reach

- **Egress control.** The domains and interfaces an agent's environment can call are restricted. This removes the external-communication leg for most agents.
- **Sandboxed code execution.** Any code an agent generates runs in an isolated environment without production credentials (OWASP ASI05).
- **Budgets and rate limits.** Calls, records touched and spend are capped per task, so a looping or hijacked agent fails small.
- **Kill switch.** Owners and security staff can suspend an agent's identity at once; with short-lived tokens, suspension takes effect within minutes (Section 3.3).
- **Memory hygiene.** What an agent may write into long-term memory or a retrieval index is controlled, since poisoned memory affects every later task (OWASP ASI06).

4.4 Designing agents that resist injection

Research since 2025 has moved from detecting injections towards designing agents whose structure limits what an injection can do. Beurer-Kellner and colleagues set out a family of design patterns built on one principle: once an agent has read untrusted input, it must be constrained so that the input cannot trigger consequential actions [15]. Their patterns trade some generality for security, and their case studies show that they remain useful for defined tasks [15].

CaMeL, from Google DeepMind and ETH Zurich, applies the same idea with software-security tools. It extracts the control flow and data flow from the user's trusted request, so that untrusted data retrieved later cannot change which steps run, and it attaches capabilities to each value to stop private data leaving through unauthorized flows [16]. On the AgentDojo benchmark it solved 77% of tasks with provable security, against 84% for an undefended system [16].

For a small or mid-sized firm the practical lesson is to prefer agents with a fixed plan for each process. An order-entry agent whose steps are fixed in advance, and which treats the purchase-order text only as data to extract, is far harder to hijack than a general assistant asked to handle the orders inbox.

5. Audit Trails and Observability

An audit trail is what turns an agent from a black box into an accountable actor. For any action it must answer five questions: which agent acted, for whom, what it did, why it did it, and who

approved it. The NIST concept paper names auditing and non-repudiation alongside authentication and authorization as core requirements for agent identity [5, 12].

5.1 What each record should hold

Table 3 lists the minimum content of an agent audit record.

Table 3. Minimum content of an agent audit record.

Field	Purpose
Timestamp and trace ID	Order events and join records across agent, gateway and server
Agent identity and version	Which agent, which configuration, which model version
On-behalf-of user	Whose authority the action used
Tool name and validated parameters	What was attempted
Policy decision and rule	Why it was allowed or denied
Approver and decision time	Human accountability for consequential actions
Result summary or hash	What changed, without storing sensitive payloads in clear
Source references	Which documents or records informed the action, to trace an injection

Author's specification, informed by [5, 12, 17].

The source-reference field is the one most often missing and the most useful in an investigation: when an agent misbehaves, the first question is what it read just before.

5.2 Tracing across systems

Agent workflows cross several components, so the log must follow one request end to end. The 2026-07-28 MCP revision cycle standardized W3C Trace Context fields in request metadata, which lets tool calls be traced across agent, gateway and server with common

observability back ends [17]. Firms should require their MCP servers and gateway to propagate these identifiers.

5.3 Integrity and retention

- Audit records are written to an append-only store that the agent's own identity cannot modify or delete.
- Retention follows the records the action affects: a purchase-order action follows financial-record retention; a quality action follows the quality system's.
- A sample of automatic actions is reviewed each week, together with every denied or failed high-risk call. A rising denial rate often signals an injection attempt or a mis-scoped tool.

5.4 Worked example: investigating a suspect action

Suppose the purchasing actor agent of Figure 1 sends a supplier a draft that quotes an unusual discount. The audit records allow the investigation to proceed in a fixed order. First, the trace identifier of the email call leads to the approval record, which shows which buyer released the draft and when. Second, the source references on the actor agent's drafting call show which summary from the reader agent it used, and the reader agent's own record shows which incoming message that summary came from. Third, the message itself can be inspected for hidden instructions. If it contains them, the reader agent behaved as designed, since it could only summarize, and the weak point was the human approval step, which is then strengthened by showing the buyer the original source alongside each draft. If it does not, the fault lies in the actor agent's instructions or tools and is fixed there. Either way, the investigation takes minutes rather than days, because each step follows a recorded link rather than a reconstruction from memory. Without source references and approval records, the same question can usually not be answered at all.

6. Governance Framework and Control Checklist

Governance makes the controls above hold as the number of agents grows. Identity vendors report that organizations quickly accumulate agents built by different teams on different platforms, many unknown to security staff [4]; OWASP's working group found the same pattern [8]. The first governance control is therefore an inventory.

6.1 The agent register

One register lists every agent, sanctioned or discovered, with its owner, purpose, identity, the tools and data it can reach, its risk tier, the users it acts for, its approval date and its next review date. An agent that is not in the register should not be able to obtain credentials. Agents discovered outside the register, such as an assistant a team connected to a shared mailbox on its own, are either brought into it through the lifecycle below or switched off. For a small firm this can start as a controlled spreadsheet; the discipline matters more than the tool.

6.2 Lifecycle

1. **Request:** the business owner states purpose, tools, data and expected autonomy.
2. **Threat review:** the design is checked against the trifecta and the OWASP list; an agent that holds all three trifecta legs is split.
3. **Approve and provision:** the identity, tool allow-list and risk tiers are created; credentials are issued through the gateway only.
4. **Operate:** denials, approvals and anomalies are monitored, and automatic actions are sample-reviewed.
5. **Review:** at least quarterly, and after any change of model, tools or owner.
6. **Retire:** the identity is revoked, tool access removed and the audit trail archived.

6.3 Control checklist

Table 4 summarizes the controls as a checklist.

Table 4. Agent security control checklist.

No.	Control	Section
1	Every agent registered with an accountable owner	6.1
2	Each agent has its own identity; no shared or personal accounts	3.1
3	Delegated, audience-bound tokens; no token passthrough	3.2
4	Tokens short-lived and issued per task; secrets kept out of context	3.3, 3.4
5	Per-agent tool allow-lists with validated parameters	4.1
6	No agent holds all three trifecta legs	2.2, 4.1
7	Human approval for consequential and irreversible actions	4.2
8	Egress limits, sandboxed code, budgets and a kill switch	4.3
9	Third-party MCP servers from an approved, version-pinned registry	2.4
10	Complete, append-only audit records with source references and trace IDs	5

Author's summary of Sections 2 to 6.

7. Conclusion

Prompt injection is not a flaw that a better prompt will fix; it follows from how language models read text [7]. Firms should therefore assume that any agent exposed to outside content can be steered, and design so that steering cannot cause serious harm. The dependable levers are outside the model: a distinct identity for each agent, delegated and short-lived credentials, narrow tools, approval gates for consequential actions, structures that keep untrusted content away from consequential decisions, and an audit record that ties every action to an agent, a user and a source. These are familiar IAM disciplines applied to a new kind of actor, which is why identity and governance, more than any model feature, are the real control point for agentic AI.

References

1. Do, V., Sood, S., & Johnston, C. (2026, June 11). Analysis on artificial intelligence use by businesses in Canada, second quarter of 2026 (Catalogue no. 11-621-M). Statistics Canada. www150.statcan.gc.ca/n1/pub/11-621-m/11-621-m2026010-eng.htm
2. Jitterbit. (2026). The state of agentic AI and automation: 1,500+ businesses tell all. www.jitterbit.com/blog/the-state-of-agentic-ai-automation-1500-businesses-tell-a
3. Outlook Business. (2025, June). Over 40% of agentic AI projects will be scrapped by 2027, says Gartner. *Outlook Business*. www.outlookbusiness.com/amp/story/deeptech/artificial-intelligence/over-40-of-ag
4. Biometric Update. (2026, June). Identity control plane emerges as next battleground for AI agents. *Biometric Update*. www.biometricupdate.com/202606/identity-control-plane-emerges-as-next-battlegrou
5. National Institute of Standards and Technology. (2026, February 5). New concept paper on identity and authority of software agents. www.nist.gov/news-events/news/2026/02/new-concept-paper-identity-and-authority-s
6. Booth, H., Fisher, W., Galluzzo, R., & Roberts, J. (2026). Accelerating the adoption of software and artificial intelligence agent identity and authorization [Concept paper]. NIST National Cybersecurity Center of Excellence. www.nccoe.nist.gov/sites/default/files/2026-02/accelerating-the-adoption-of-soft
7. Willison, S. (2025, June 16). The lethal trifecta for AI agents: Private data, untrusted content, and external communication. simonwillison.net/2025/Jun/16/the-lethal-trifecta/
8. TechRepublic. (2025, December). OWASP drops first AI agent risk list. *TechRepublic*. www.techrepublic.com/article/news-owasp-ai-agent-risk-list/
9. Cycode. (2026). OWASP Top 10 for Agentic Applications [Blog post]. cycode.com/blog/owasp-top-10-agentic-applications/
10. Model Context Protocol. (2026). Security best practices. Retrieved November 12, 2026. modelcontextprotocol.io/specification/latest/basic/security_best_practices
11. Claburn, T. (2026, July 29). MCP gets an enterprise makeover. *The Register*. www.theregister.com/ai-and-ml/2026/07/29/mcp-gets-an-enterprise-makeover/5280027
12. Biometric Update. (2026, March). NIST concept paper explores identity and authorization controls for AI agents. *Biometric Update*. www.biometricupdate.com/202603/nist-concept-paper-explores-identity-and-authoriz
13. Jones, M., Nadalin, A., Campbell, B., Bradley, J., & Mortimore, C. (2020). OAuth 2.0 Token Exchange (RFC 8693). Internet Engineering Task Force. datatracker.ietf.org/doc/html/rfc8693

14. Meyer zu Selhausen, K., & Fett, D. (2022). OAuth 2.0 Authorization Server Issuer Identification (RFC 9207). Internet Engineering Task Force. datatracker.ietf.org/doc/html/rfc9207

15. Beurer-Kellner, L., Buesser, B., Creţu, A.-M., Debenedetti, E., Dobos, D., Fabian, D., … Volhejn, V. (2025). Design patterns for securing LLM agents against prompt injections. *arXiv preprint arXiv:2506.08837*. arxiv.org/abs/2506.08837

16. Debenedetti, E., Shumailov, I., Fan, T., Hayes, J., Carlini, N., Fabian, D., … Tramèr, F. (2025). Defeating prompt injections by design. *arXiv preprint arXiv:2503.18813*. arxiv.org/abs/2503.18813

17. Vaughan, D. (2026, May 30). The MCP 2026-07-28 specification RC: What Codex CLI users need to know. codex.danielvaughan.com/2026/05/30/mcp-2026-07-28-spec-rc-stateless-protocol-cod

How to cite this article

Lee, Q. F. (2026). Agent Security and Identity: Prompt Injection, Least-Privilege Tool Access, Agent Credentials and Audit Trails. *Chmltech Online Journal*, *105*(5). https://www.chmltech.com/post/agent-security-and-identity-prompt-injection-least-privilege-credentials-audit